Student Timetable Conflict Detection – Complete Python Solution

Problem Statement:

Develop a Python program to validate a student's timetable by comparing course schedules represented using strings and tuples. Detect overlapping class periods, identify unavailable rooms, and validate whether a proposed timetable is conflict-free.

The program uses only your assigned Python syllabus concepts.

1. Computational Thinking

Input:

- Number of courses.
- Course names.
- Course days.
- Starting and ending periods.
- Room numbers.
- Unavailable rooms.
- Proposed timetable details.

Processing:

1. Read course details.
2. Store schedules using tuples and nested lists.
3. Compare the days and periods of different courses.
4. Detect overlapping class periods.
5. Check room availability.
6. Identify timetable conflicts.
7. Validate the proposed timetable.
8. Generate the final timetable report.

Output:

- Course-wise timetable.
- Overlapping schedule details.

- Unavailable room details.
- Timetable validation status.
- Final conflict-free or invalid status.

2. Timetable Validation Rules

For this example, assume:

Rule	Condition
Schedule format	Day, start period, end period
Period range	1–8
Same-day conflict	Two courses cannot overlap
Room availability	Unavailable rooms cannot be assigned
Different days	Same period on different days is allowed
Valid schedule	No overlapping periods or unavailable rooms

Example:

Course	Day	Start	End	Room
Python	Monday	1	2	R101
Maths	Monday	2	4	R102
English	Tuesday	1	2	R103

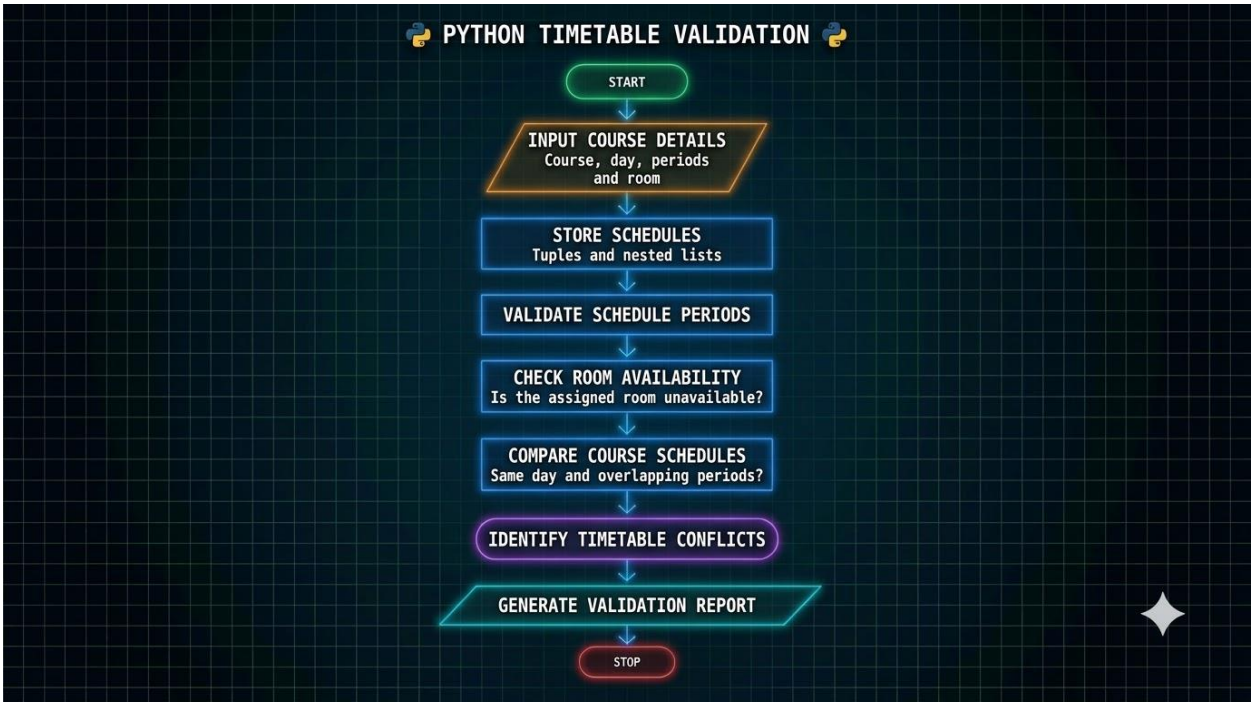
Python and Maths have a conflict because both occupy period 2 on Monday.

3. Algorithm

1. Start.
2. Read the number of courses.
3. Read course name, day, starting period, ending period, and room.
4. Store each course schedule in a tuple.
5. Store all schedules in a nested list.

6. Read unavailable room numbers.
7. Check whether each schedule has valid period values.
8. Check whether assigned rooms are unavailable.
9. Compare every pair of courses.
10. Check whether their days are the same.
11. Check whether their periods overlap.
12. Record all overlapping schedules.
13. Validate the complete timetable.
14. Display the conflict report.
15. Display the final timetable status.
16. Stop.

4. Flowchart



5. Pseudocode

START

```
INPUT number_of_courses

schedules = empty list
conflicts = empty list

FOR each course:

    INPUT course_name
    INPUT day
    INPUT start_period
    INPUT end_period
    INPUT room

    STORE course details in tuple
    APPEND tuple to schedules

INPUT unavailable_rooms

FOR each schedule:

    CHECK period validity

    IF room is in unavailable_rooms:
        RECORD room conflict

FOR i = 0 TO number_of_courses - 1:
```

```
    FOR j = i + 1 TO number_of_courses - 1:

        IF schedules[i].day == schedules[j].day:

            IF schedules[i].start <= schedules[j].end
               AND schedules[j].start <= schedules[i].end:

                RECORD timetable conflict

IF conflicts is empty:
    DISPLAY "TIMETABLE VALID"

ELSE:
    DISPLAY all conflicts
    DISPLAY "TIMETABLE INVALID"

STOP
```

6. Complete Python Program

```
# Student Timetable Conflict Detection

# Input number of courses
number_of_courses = int(input("Enter number of courses: "))

# Store course schedules
# Tuple format: (Course, Day, Start, End, Room)
```

```
schedules = []

# Input course details
for i in range(number_of_courses):

    print("\nEnter Course", i + 1, "Details")

    course_name = input("Enter course name: ").strip()
    day = input("Enter day: ").strip().lower()

    start_period = int(input("Enter starting period: "))
    end_period = int(input("Enter ending period: "))

    room = input("Enter room number: ").strip().upper()

    # Validate schedule
    if (start_period < 1 or end_period > 8 or
            start_period > end_period):

        print("Invalid period details!")

    else:

        # Store details as a tuple
        course = (
```

```
            course_name,
            day,
            start_period,
            end_period,
            room
        )

        schedules.append(course)


# Input unavailable rooms
unavailable_count = int(input("\nEnter number of unavailable rooms: "))

unavailable_rooms = []

for i in range(unavailable_count):

    room = input("Enter unavailable room: ").strip().upper()

    unavailable_rooms.append(room)


# Store all conflicts
conflicts = []

# Check room availability
```

```
for course in schedules:

    course_name, day, start, end, room = course

    if room in unavailable_rooms:

        conflict = [
            course_name,
            "ROOM UNAVAILABLE",
            room
        ]

        conflicts.append(conflict)



# Check overlapping course periods
for i in range(len(schedules)):

    for j in range(i + 1, len(schedules)):

        # Tuple unpacking
        course1, day1, start1, end1, room1 = schedules[i]

        course2, day2, start2, end2, room2 = schedules[j]

        # Check same day
```

```
        if day1 == day2:

            # Check period overlap
            if start1 <= end2 and start2 <= end1:

                conflict = [
                    course1,
                    course2,
                    "TIME OVERLAP",
                    day1
                ]

                conflicts.append(conflict)



# Display timetable
print("\n==============================================")
print("           STUDENT TIMETABLE")
print("==============================================")

print("Course      Day      Start   End   Room")
print("----------------------------------------------")

for course in schedules:

    name, day, start, end, room = course
```

```
    print(name, day, start, end, room)



# Display conflict report
print("\n============================================")
print("            CONFLICT REPORT")
print("============================================")

if len(conflicts) == 0:

    print("No timetable conflicts detected.")

else:

    for conflict in conflicts:

        print(conflict)



# Final timetable validation
print("\n============================================")

if len(conflicts) == 0:

    print("TIMETABLE STATUS: VALID")
```

else:

print("TIMETABLE STATUS: INVALID")

print("==")

print(" VALIDATION COMPLETED")

7. Sample Input

Enter number of courses: 4

Enter Course 1 Details

Enter course name: Python

Enter day: Monday

Enter starting period: 1

Enter ending period: 2

Enter room number: R101

Enter Course 2 Details

Enter course name: Maths

Enter day: Monday

Enter starting period: 2

Enter ending period: 4

Enter room number: R102

Enter Course 3 Details

Enter course name: English

Enter day: Tuesday

Enter starting period: 1

Enter ending period: 2

Enter room number: R103

Enter Course 4 Details

Enter course name: Physics

Enter day: Wednesday

Enter starting period: 3

Enter ending period: 4

Enter room number: R104

Enter number of unavailable rooms: 1

8. Sample Output

```
==============================================
          STUDENT TIMETABLE
==============================================

Course      Day      Start   End   Room
----------------------------------------------
Python      monday    1      2     R101
Maths       monday    2      4     R102
English     tuesday   1      2     R103
Physics     wednesday 3      4     R104
```

==========

CONFLICT REPORT

==========

['Physics', 'ROOM UNAVAILABLE', 'R104']

['Python', 'Maths', 'TIME OVERLAP', 'monday']

==========

TIMETABLE STATUS: INVALID

==========

VALIDATION COMPLETED

9. Overlapping Period Calculation

The important condition for detecting overlapping periods is:

$$ Start_1 \leq End_2 $$

AND

$$ Start_2 \leq End_1 $$

Both conditions must be true, and the course days must match.

Example:

Course	Start	End
Python	1	3
Maths	3	5

Both courses overlap at period 3.

Non-overlapping example:

Course	Start	End
Python	1	2
Maths	3	5

There is no conflict because the periods do not overlap.

10. Additional Test Cases

Situation	Expected result
Two courses on different days	No time conflict
Two courses overlap on same day	Time conflict
Same starting and ending period	Conflict
Assigned room is unavailable	Room conflict
All schedules are valid	Valid timetable
Starting period greater than ending period	Invalid schedule
Period greater than 8	Invalid schedule

11. Assigned Python Syllabus Concepts Used

Syllabus concept	Application
Computational thinking	Breaking timetable validation into steps
Algorithm	Step-by-step conflict detection
Flowchart	Program execution flow
Pseudocode	Logical representation
Variables	Course names, days, periods
Data types	Strings and integers
Operators	==, <=, >=, and
Conditional statements	if, else

Loops	for and nested for
Strings	Course names, days and room numbers
Lists	Store course schedules
Nested lists	Store conflict details
Tuples	Store individual course schedules
Tuple operations	Tuple indexing and comparison
Tuple unpacking	Extract course schedule details

Complete syllabus-based solution

Important note: This example treats the starting and ending periods as inclusive. Therefore, a class scheduled from period 1 to period 3 occupies all three periods. It checks time conflicts and room unavailability, but does not separately check whether two non-overlapping courses are assigned to the same room at the same time.